Automatisk fornyelse af Pinterest-tokenet

Workflow: .github/workflows/niche-scan.yml · værktøj: tools/niche_scanner.py · forgænger: niche-scan-ci-2026-11-03.md

Pinterest access-token holder **30 dage** (`expires_in` 2.592.000 s), og indtil nu var fornyelse en manuel opgave hver måned. Fra 3/11 2026 fornyer den natlige kørsel det selv: svarer Pinterest 401, byttes refresh-tokenet til et nyt access-token, kaldet gentages, og det nye par lægges tilbage i repository-secrets. Ingen tokenværdi forlader runneren.

Hvad kørslen gør nu

Trin	Handling
1	Venter til kl. 02:30 dansk tid (uændret sommertids-gate)
2	Tjekker om der enten er et access-token eller et komplet sæt refresh-oplysninger. Intet af det: ::notice, ingen scanning, ingen måledag
3	Kører tools/test_niche_scanner.py (48 tests, offline)
4	Scanner med --token-out "$RUNNER_TEMP/pinterest-tokens.json" — filen findes kun hvis der blev fornyet noget
5	Er filen skrevet: begge tokens maskeres med ::add-mask::, lægges i secrets med gh secret set, og filen slettes
6	Måledage i job-summaryen, commit og push af reports/

Trin 5 kører med `always()`, så et roteret refresh-token også bliver gemt når scanningen fejler bagefter. Det er ikke kosmetik: et roteret refresh-token er brugt op, og det nye findes kun i den fil.

De fem secrets

Secret	Indhold	Hvor den kommer fra
PINTEREST_ACCESS_TOKEN	nuværende access-token	workflowet selv, efter første fornyelse
PINTEREST_REFRESH_TOKEN	refresh-token	OAuth-flowet — og derefter workflowet
PINTEREST_CLIENT_ID	appens id	developers.pinterest.com → app
PINTEREST_CLIENT_SECRET	appens secret	samme sted — kun som secret, aldrig i repoet
PINTEREST_SECRETS_TOKEN	fine-grained PAT med Secrets: Read and write	ejerens GitHub-indstillinger

De fire første er nok til den første kørsel; den femte er det der gør fornyelsen holdbar. `GITHUB_TOKEN` kan ikke skrive secrets — derfor en PAT, og den bør være begrænset til dette repository og til Secrets-tilladelsen alene.

Rotation: det gamle refresh-token dør

Pinterest v5 roterer refresh-tokenet ved **hver** fornyelse: svaret indeholder et nyt, og det man sendte er brugt op i samme øjeblik. Det giver to regler, som koden følger:

1. **Gem før brug.** `pinterest_refresher()` skriver det nye par til `--token-out` (atomisk, mode 600) «før» den returnerer access-tokenet. Fejler gemningen, fejler fornyelsen — og scanningen — i stedet for at fortsætte med et token ingen har gemt.
2. **Forny ikke uden et sted at gemme.** Er `--token-out` ikke givet, returnerer `pinterest_refresher()` `None`, og et 401 bliver en fejl i rapporten. Ellers ville en kørsel brænde

refresh-tokenet og efterlade den næste nat uden adgang.

Der er også en anden vej ind: mangler `PINTEREST_ACCESS_TOKEN` helt, men refresh-oplysningerne er der, henter scanneren et access-token før det første Trends-kald. Så er secret-datoen ikke længere et stop for scanningen.

Hvorfor kun 401 fornyes

Svar	Betydning	Kørslen gør
200	token virker	scanner
401	tokenet er udløbet	fornyer én gang, gentager kaldet
403	appen/adgangen må ikke kalde endpointet	fejl i rapporten, ingen fornyelse
429 / 5xx	rate eller midlertidig fejl	tre genforsøg med backoff (uændret)
400 på /v5/oauth/token	refresh-tokenet er dødt (invalid_grant)	fejl med henvisning til manuel re-autorisering

En 403 er manglende rettigheder, ikke et udløbet token. At fornye på den ville brænde et rotation uden at ændre svaret — og gøre det sværere at se hvad der faktisk er galt.

Grænsen ved 60 dage

`refresh_token_expires_in` er 5.184.000 s, altså 60 dage. Ifølge en rapport fra fællesskabet (n8n-forum, 19/9 2026) tælles de 60 dage fra den **oprindelige autorisering** — rotationen forlænger dem ikke, og derefter hjælper hverken fornyelse eller nye secrets: kun ejerens fulde re-autorisering. Det er ikke Pinters egen dokumentation, men påstanden er værd at tage alvorligt, for den fejl den beskriver er tavs.

Konsekvensen for os: den automatiske fornyelse fjerner den månedlige 30-dages-opgave, men ikke en re-autorisering hvert andet måned. Den bør stå i ejerens kalender — og hvis 60-dages-grænsen viser sig at gælde, fejler kørslen rødt med `invalid_grant` i stedet for at gå stille i stå.

Hvad der ikke kan ses af koden

- **Om fornyelsen virker mod den ægte API.** Der findes intet `client_id`, `client_secret` eller refresh-token endnu, så `POST /v5/oauth/token` er ikke kaldt. Requests opbygning (Basic-auth med `client_id:client_secret`, `grant_type=refresh_token`, form-body) er dækket af en test mod en falsk transport, ikke af et rigtigt svar.
- **Om 60-dages-grænsen gælder** som rapporten siger. Det afgøres først ved anden fornyelse.
- **Hvor længe et gemt token holder.** Access-tokenet gemmes med i secrets, så de fleste nætter slet ikke fornyer; men hvor tit `gh secret set` må skrive, afhænger af om Pinterest holder de 30 dage.

Verifikation

- `python3 tools/test_niche_scanner.py` → **48 tests, OK** (40 før). De otte nye dækker Basic-auth og form-body, svar uden `access_token`, at fornyelse kræver alle fire dele, at det roterede token gemmes med mode 600 før retur, 401 → forny → gentag, at 403 **ikke** fornyes, fejlet fornyelse som rapportfejl, og bootstrap uden access-token.
- Workflowen er parset som YAML, og alle `run:`-blokke er tjekket med `bash -n`.
- Trin 5 er kørt isoleret mod en falsk `gh` i tre tilfælde: med PAT (to `secret set`-kald), uden PAT (rød annotation, exit 1) og uden fornyelse (ingen skrivning).
- **Ikke afprøvet:** et rigtigt Pinterest-kald — hverken Trends eller `/v5/oauth/token`.

Næste skridt

1. Ejer: app-review til Standard-adgang, og læg `PINTEREST_CLIENT_ID`, `PINTEREST_CLIENT_SECRET` og `PINTEREST_REFRESH_TOKEN` som secrets.
2. Ejer: opret en fine-grained PAT med Secrets: Read and write til dette repository og læg den som `PINTEREST_SECRETS_TOKEN`.
3. Første nat: kørslen henter et access-token med refresh-tokenet og gemmer det selv.
4. Sæt en påmindelse om fuld re-autorisering ca. hver 60. dag, indtil 60-dages-grænsen er målt.